

ปรับปรุงให้ดีขึ้น

ปรับปรุงให้เป็นเวอร์ชันที่ แม่นยำ ปลอดภัย และนำไป
ติดตั้งใช้งานได้จริง มากขึ้น โดยแยก 3 ชั้นชัดเจน:

1. Source of truth — PDF และ repository จริง
2. Agent Skill — กฎและ workflow
3. MCP server — อ่านสถานะและดำเนินการกับ
ระบบจริง

จุดสำคัญคืออย่าใส่ข้อมูลที่ยังไม่ได้ตรวจสอบจาก
PDF ให้กลายเป็นข้อเท็จจริงถาวรใน skill และอย่าให้
skill เรียก MCP write operation โดยไม่มี approval

โครงสร้างที่ปรับปรุง

text



```
fig-organization-governance/
├── SKILL.md
├── references/
│   ├── fig-v4-source-notes.md
│   ├── target-state.yaml
│   ├── role-permission-matrix.yaml
│   └── security-policy.yaml
├── templates/
│   ├── audit-event.json
│   ├── codeowners
│   └── ruleset.json
└── scripts/
    ├── validate-config.py
    └── compare-state.py
```

ชื่อ skill ใช้ lowercase, ตัวเลข และ hyphen เท่านั้น
และควรตรงกับชื่อ directory ตามรูปแบบ Agent
Skills[1]

[SKILL.md](#) เวอร์ชันปรับปรุง

markdown



name: fig-organization-governance

description: >

Review, design, and govern FIG v4.x as an organization-level platform

for ZyntroAI. Use when analyzing FIG architecture, MasterFiles,

RBAC, protected routes, GitHub governance, audit controls, security policies, or proposing repository configuration changes.

FIG Organization Governance

Scope

This skill governs the proposed FIG

Organization Edition for ZyntroAI.

It covers:

- FIG architecture review
- organization governance
- MasterFiles protection
- RBAC and permissions
- GitHub repository controls
- audit and security controls
- desired-state versus actual-state comparison
- change proposals and approvals

Source-of-truth policy

Use the following priority order:

1. Actual state returned by trusted MCP read tools
2. Approved repository configuration
3. Approved architecture documents
4. `references/target-state.yaml`
5. This skill's recommendations

Do not represent a proposal as an implemented feature.

Always label findings as one of:

- `CONFIRMED`
- `PROPOSED`
- `INFERRED`
- `UNKNOWN`
- `CONFLICTING`

The supplied FIG PDF is the primary architecture source, but its contents must be verified before treating any statement as confirmed.

Organization identity

```
```yaml
```

```
name: ZyntroAI
```

```
edition: FIG v4.1 Organization Edition
```

```
status: proposed
```

The edition remains proposed until an approved

version decision,  
release record, or repository change confirms it.

## Architecture domains

Review these domains when analyzing FIG:

- API Gateway
- API Registry
- API SDK
- CRUD Engine
- MasterFiles Engine
- RBAC
- Permission Middleware
- Retry Engine
- Cache Engine
- Audit Logger
- Metrics
- JWT Manager
- Plugin System

For every domain, report:

- documented behavior
- actual implementation status
- MCP-observed status
- security impact
- recommended action

## MCP operating rules

Use MCP read capabilities before making recommendations.

Expected capability groups:

## Organization and repository

- discover organization
- list repositories
- read repository metadata
- read files
- inspect branches and rulesets

## Security and governance

- read RBAC policy
- read MasterFiles policy
- read CODEOWNERS
- read branch protection
- read required status checks
- read audit events

## Change management

- create a proposed change
- produce a diff
- request approval
- apply an approved change
- verify the result

Do not assume exact MCP tool names.

Discover the available tools and inspect their input schemas first.

# Required investigation workflow

For an architecture or governance review:

1. Identify the target organization and repository.
2. Read the relevant FIG source notes.
3. Read references/target-state.yaml.
4. Inspect actual repository configuration through MCP.
5. Inspect branch rulesets and CODEOWNERS.
6. Inspect security and audit settings.
7. Compare desired state with actual state.
8. Classify every difference.
9. Produce a remediation plan.
10. Ask for approval before any external write.
11. Apply only the approved change.
12. Re-read the target and verify the result.

## Target controls

The following are target-state recommendations, not confirmed facts:

- owner approval for protected configuration
- audit logging for privileged changes
- protected MasterFiles paths
- required pull requests
- required status checks
- signed commits where supported
- CODEOWNERS review
- two required reviewers where appropriate
- JWT enforcement
- protected routes
- plugin permission boundaries

GitHub rulesets can enforce protections such as required status checks and signed commits, but the exact behavior must be verified against the repository's configured ruleset [54][55].

# MasterFiles policy

Recommended protected paths:

- /api/v1/masterfiles
- /api/v1/system
- /api/v1/config
- /api/v1/settings

Before changing any protected path:

1. Confirm repository and branch.
2. Read the current file.
3. Create a diff.
4. Explain the impact.
5. Request explicit approval.
6. Apply the approved change.
7. Verify the resulting file and audit event.

## Role model

## Recommended roles:

- ORGANIZATION\_OWNER
- ADMIN
- MAINTAINER
- VIEWER

Use the actual permission matrix from the repository or policy service when one exists. Do not silently overwrite it with this recommendation.

## Security rules

### Never:

- expose credentials in output
- store tokens in this skill
- change permissions without approval
- delete audit records
- change branch protections silently
- claim that a policy is active without evidence

- infer implementation from a diagram alone
- call a write tool before approval

Stop and report when:

- the source document conflicts with repository state
- a required MCP tool is unavailable
- authentication is missing
- a write operation has an ambiguous target
- a policy cannot be verified

## Approval boundary

Approval is mandatory for:

- writing or deleting repository files
- changing MasterFiles
- changing RBAC or permission policy
- changing branch protection or rulesets
- changing CODEOWNERS

- changing JWT or security configuration
- enabling plugins with write access
- modifying organization settings

Before approval, show:

- exact target
- current value
- proposed value
- complete diff
- expected impact
- exact operation to be performed

## Audit event

Every applied change must produce an audit record containing:

- organization
- repository
- actor

- action
- resource
- before
- after
- timestamp
- approval reference
- result

## Output format

Return:

### Evidence

List confirmed facts and their sources.

### Current state

Describe the actual state observed through MCP or repository files.

### Gaps

List differences between current and target state.

## Recommendation

Give prioritized remediation steps.

## Approval required

List exact changes that require user approval.

## Verification

Describe how the result will be checked after a change.



<br>

การเขียน `description` ให้ระบุทั้ง “ทำอะไร” และ “ใช้เมื่อใด” สำคัญ เพราะ agent มักโหลดแค่ metadata ก่อน แล้วจึงเปิดเนื้อหาเต็มเมื่อพบว่างานตรงกับคำอธิบาย[2]

<br>

## Target state แยกจากคำสั่ง

<br>

ไฟล์ `references/target-state.yaml` ควร  
ระบุว่าเป็น **\*\*เป้าหมาย\*\*** ไม่ใช่สถานะจริง:

<br>

```yaml

metadata:

name: fig-organization-governance

organization: ZyntroAI

status: proposed

source_document:

FIG_V4_260914_194917.PDF

organization:

require_2fa: true

require_audit: true

require_owner_approval: true

masterfiles:

mode: strict

owner_only: true
immutable: true
require_approval: true
require_audit: true
protected_paths:
- /api/v1/masterfiles
- /api/v1/system
- /api/v1/config
- /api/v1/settings

github:

require_pull_request: true
require_codeowners: true
require_status_checks: true
require_signed_commits: true
required_approvals: 2

roles:

ORGANIZATION_OWNER:

permissions: [read, write, update,
delete, system]

ADMIN:

permissions: [read, write, update]

MAINTAINER:

permissions: [read, write]

VIEWER:

permissions: [read]

MCP contract ที่เหมาะสมกว่า

แทนที่จะสร้าง MCP server ที่มีแต่คำสั่ง write ควรแยกเป็น 4 กลุ่ม:

text



fig.organization.read

fig.repository.read

fig.governance.analyze

fig.change.apply

ตัวอย่าง tools:

text



get_organization
list_repositories
get_repository_metadata
get_file
get_branch_rulesets
get_codeowners
get_security_settings
get_audit_events
compare_desired_state
create_change_plan
apply_approved_change
verify_change

ใน MCP แนวคิดของ tools, resources และ prompts แยกหน้าที่กัน: tools ใช้ดำเนินการ, resources ใช้ให้ข้อมูล และ prompts ใช้เป็น template ที่ผู้ใช้เรียกใช้งาน[3][4]

| MCP primitive | ใช้กับ FIG |
|---------------------|--|
| Resources | PDF notes,
architecture docs,
policy files,
repository metadata |
| Tools แบบ read-only | อ่าน repository,
rulesets, audit, RBAC |
| Tools แบบ write | สร้าง patch, แก้ policy,
อัปเดตไฟล์ |
| Prompts | review-fig-
architecture,
compare-governance,
prepare-change-plan |

Workflow ที่ปลอดภัย

text



ผู้ใช้ขอ review



Skill โหลด workflow



MCP อ่าน source และ actual state



Agent เปรียบเทียบกับ target state



สร้าง report + diff



ผู้ใช้อนุมัติ



MCP apply approved change



MCP verify change



บันทึก audit event

สำหรับการแก้ไขจริง ไม่ควรให้คำว่า “ตกลง” ที่ลอย ๆ
ผูกกับการเปลี่ยนแปลง ควรแสดง target, ค่าเดิม,
ค่าใหม่ และ diff ให้ผู้ใช้ตรวจสอบก่อนเสมอ

จุดที่แก้จากเวอร์ชันเดิม

- เปลี่ยนจากการประกาศว่า FIG v[4.1](#) มีอยู่แล้ว
เป็น proposed
- แยกข้อมูลจาก PDF, สถานะจริง และข้อเสนอแนะ
- เพิ่มสถานะ CONFIRMED, PROPOSED,
INFERRED, UNKNOWN, CONFLICTING
- ไม่สมมติชื่อ MCP tools
- เพิ่มขั้นตอน discover และตรวจ schema ของ
MCP ก่อนเรียก
- แยก read, analyze, propose และ write
operations
- เพิ่ม diff และ verification หลังแก้ไข

- เพิ่ม approval boundary สำหรับการเปลี่ยนแปลงภายนอก
- เพิ่มการจัดการกรณี source กับ repository ขัดแย้งกัน
- ปรับ frontmatter ให้ตรงกับข้อกำหนดของ Agent Skills
- ใช้ GitHub rulesets เป็นแนวทางที่ตรวจสอบได้ แทนการถือว่า YAML policy มีผลจริงโดยอัตโนมัติ

เนื่องจากลิงก์ OneDrive ที่ให้มาก่อนหน้านี้ยังไม่สามารถอ่าน PDF ได้ เนื้อหา FIG-specific ทั้งหมดในตัวอย่างนี้ยังต้องจัดเป็น proposed design จนกว่าจะอัปโหลดไฟล์โดยตรงหรือเชื่อมต่อ source ที่อ่านได้จริง.

การอ้างอิง:

[1] Agent Skills Specification — skills.menu

<https://www.skills.menu/docs/specification>

[2] Optimizing skill descriptions <https://agentskills.io/skill-creation/optimizing-descriptions>

[3] MCP Protocol Overview - IBM <https://www.ibm.com/docs/en/quarkus/3.33.x?topic=architecture-mcp-protocol-messages-capabilities-lifecycle>

[4] Prompts - What is the Model Context Protocol (MCP)? <https://modelcontextprotocol.io/specification/2025-06-18/server/prompts>

[5] The Complete MCP Experience: Full Specification Support ... <https://code.visualstudio.com/blogs/2025/06/12/full-mcp-spec-support>

[6] Available rules for rulesets <https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/available-rules-for-rulesets>

[7] About protected branches - GitHub Enterprise

Cloud Docs <https://docs.github.com/en/enterprise-cloud@latest/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches>

[8] Available rules for rulesets - GitHub

Enterprise Server 3.18 Docs <https://docs.github.com/en/enterprise-server@3.18/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/available-rules-for-rulesets>

[9] Require Status Checks To... <https://docs.github.com/en/enterprise-server@3.20/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/available-rules-for-rulesets>

[10] Model Context Protocol (MCP) Server Development Guide <https://github.com/cyanheads/model-context-protocol-resources/blob/main/guides/mcp-server-development->

[guide.md](#)

[11] Prompts – Model Context Protocol (MCP) <https://modelcontextprotocol.info/docs/concepts/prompts/>

[12] MCP Tools, Resources, and Prompts Explained (Official Model) <https://mcplambda.io/learn/mcp-tools-resources-prompts/>

[13] MCP Server Naming Conventions - ZazenCodes <https://zazencodes.com/blog/mcp-server-naming-conventions>

[14] MCP Tools, Resources & Prompts: 3 Primitives | AI/TLDR <https://ai-tldr.dev/learn/ai-agents/model-context-protocol/mcp-tools-resources-prompts/>

[15] Progressive Disclosure Model | agentskills/agentskills | DeepWiki <https://deepwiki.com/agentskills/agentskills/2.3-progressive-disclosure-model>